
PortfolioOptimisers.jl

Daniel Celis Garza¹ 

¹Independent researcher, Oxford, UK

July 15, 2026

ABSTRACT

Portfolio optimisation is the science of either: 1) Minimising risk whilst keeping returns to acceptable levels. 2) Maximising returns whilst keeping risk to acceptable levels. To some definition of acceptable, and with any number of additional constraints available to the optimisation type. There exist myriad statistical, pre- and post-processing, optimisations, and constraints that allow one to explore an extensive landscape of “optimal” portfolios. PortfolioOptimisers.jl is an attempt at providing as many of these as possible under a single banner and making it accessible to all. We make extensive use of Julia’s type system, module extensions, and multiple dispatch to simplify development and maintenance, while keeping robustness, testability, and usability high.

Keywords. Portfolio Optimisation · Quantitative Investment · Conic Optimisation · Parameter Estimation

1 Introduction

The field of portfolio optimisation was introduced by Harry Markowitz’s seminal 1952 paper “Modern portfolio theory” [1]. However, the field has grown significantly since then, with many new techniques and methods being developed. The original Markowitz model is highly sensitive to estimation errors in the input parameters, particularly the expected returns. After all, it attempts to summarise the entire distribution of returns in highly compressed summary statistics, both of which can be sensitive to outliers and atypical conditions for the lookback period. Many have studied and addressed the strengths and weaknesses of the Markowitz model [2], [3], [4].

Unfortunately, most implementations live in disparate, unconnected, often proprietary, bespoke codebases, which limit their applicability. It is only recently that various advanced yet very usable libraries have been published [5], [6], [7]. However, each has its own strengths, weaknesses, scope, and idiosyncrasies. That is no different from PortfolioOptimisers.jl, but it is our hope that this library serves as a unifying framework for the various techniques and methods available in the field, while also eventually providing a simple and intuitive interface for users, with advanced features for experts.

There also exist myriad optimisation methods, pre-filtering, distribution and moment estimators, machine learning techniques, validation, and parameter tuning methods that can all be used together to improve the out-of sample performance of a portfolio. To this day, only [7] has succeeded in providing a unified framework for this. `PortfolioOptimisers.jl` provides an alternative that is not tied to the scikit-learn [8] or cvxpy [9], [10] APIs and ecosystems, as well as providing different functionality and a different architectural philosophy.

2 Design and implementation

2.1 Basic example

`PortfolioOptimisers.jl` is built with modularity and extensibility in mind. We can demonstrate a simple improvement over the Markowitz model by using by adding an L2 regularisation term to the optimisation problem.

$$\begin{aligned}
 \min_{\mathbf{w}} \quad & \mathbf{w}^\top \Sigma \mathbf{w} + \lambda \|\mathbf{w}\|_2^2 \\
 \text{s.t.} \quad & \mathbf{w}^\top \boldsymbol{\mu} \geq \mu_i \forall i \in \{1, \dots, N\} \\
 & \mathbf{w}^\top \mathbf{1} = 1 \\
 & \mathbf{w} \geq 0 \\
 & w_{i \neq \text{AAPL}} \leq 1 \\
 & w_{\text{AAPL}} \leq 0.2
 \end{aligned} \tag{2.2}$$

```

# Import packages.
using PortfolioOptimisers, CSV, TimeSeries, Clarabel, StatsPlots, GraphRecipes
# Load data.
X = TimeArray(CSV.File(joinpath(@__DIR__, "../examples/SP500.csv.gz")));
    timestamp = :Date)
# Compute returns.
rd = prices_to_returns(X)
# Split into training and test sets.
rd_train, rd_test = train_test_split(rd; test_size = 0.2)
# Mean Risk optimisation. Defaults to minimising risk.
mr = MR(;
    # Variance (default)
    r = Variance(),
    # Configured optimiser.
    opt = JuMPOpt(;
        # Using Clarabel as the solver. It's possible to provide fallbacks in the form of

```

```

# a vector, `PortfolioOptimisers.jl` will iterate until it finds one that works
# or they all fail.
slv = Solver(; name = :clarabel, solver = Clarabel.Optimizer,
    settings = Dict("verbose" => false)
), # slv

# Weight bounds using an estimator (it builds the constraint based on the data).
# Lower bound for all assets is 0, upper bound for AAPL is 0.2, for the rest it's
# 1.
wb = WBE(; lb = 0, ub = "AAPL" => 0.2),

# This maps asset names to their indices in the data, as well as sets to which
# they belong. It is needed to build constraints from estimators and is used by
# other components such as some prior statistics.
sets = AssetSets(; dict = Dict("nx" => rd.nx)),

# L2 regularisation using a squared L2 norm with a scalar value of 0.0001. This
# is used to prevent weight concentration and thus reduce overfitting and improve
# generalisation.
l2 = L2Reg(; val = 0.01, alg = SquaredSOCRiskExpr()),

# Arithmetic returns with 100 evenly distributed points between the minimum and
# maximum returns in the training set. This way we can compute the efficient
# frontier, which is a subset of pareto fronts.
ret = ArithmeticReturn(; lb = Frontier(; N = 100))
) # opt
) # mr

# Perform optimisation on the training set.
res = optimise(mr, rd_train)

# Predict on training data.
pred_train = predict(res, rd_train)

# Predict on test data.
pred_test = predict(res, rd_test)

# Scenario based standard deviation.
r = SCM(; alg = SOCRiskExpr())

# Plot the results.
plt = plot_measures(pred_train; x = r, label = "Training", zcolor = nothing)
plt = plot_measures(pred_test; x = r, plt = plt, label = "Test", zcolor = nothing,
    markercolor = :red, ylabel = "Mean Return",
    xlabel = "Standard Deviation")
savefig(plt, joinpath(@__DIR__, "fig1.svg"))

```

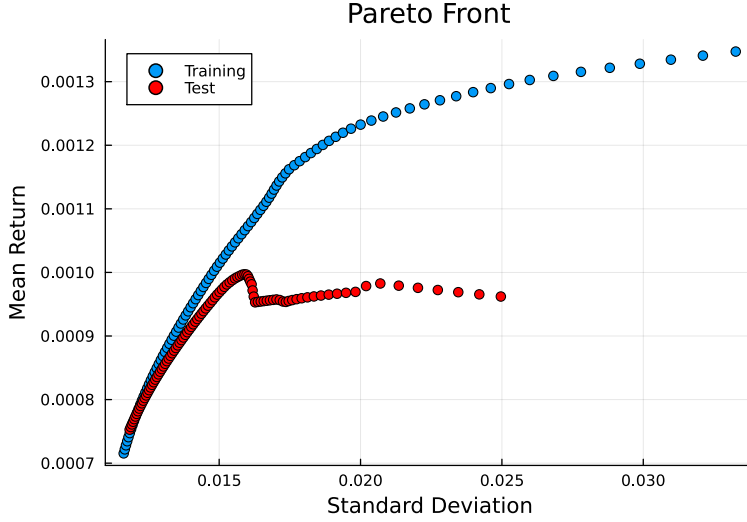


Figure 1: Train and test efficient frontier for an L2 regularised Markowitz model with a scenario-based standard deviation risk measure. The training set does much better than the test set, this is why other portfolio optimisation modalities have been invented.

2.2 Data flow

The library is configured to allow for different workflows. There is no rigid structure to the data flow. It is possible to use the library as simply as shown above or to build extremely complex workflows either manually, or using a pipeline. The functionality is grouped into three main stages, these break down in a hierarchy of standalone processes, which can be used in isolation or as part of a larger workflow.

1. Preprocessing: Data cleanup, filtering, and transformation.
 - a. Transformation: Fills out pathological data with something tractable.
 - b. Preselection: Uses various non-optimisation methods defined in the library to preselect assets based on various criteria. Can operate at the price or returns stage.
 - c. Returns computation: Computes returns but can also impute, drop, or fill missing data. Not as advanced or flexible as the transformation or preselection stages but can be used standalone.
2. Processing: These can be used as separate steps and provided precomputed (only valid for certain optimisers and in standalone optimisations), or as a single step as part of an optimisation (required for certain meta optimisers and cross-validation). The only requirement to perform an optimisation is the computation of prior statistics, and for certain optimisers, a clustering.
 - a. Prior statistics: These include standalone moment estimation, but also an integrative returns distribution estimation and adjustments to the moment estimations.

- b. Phylogeny: These uncover the relational structure of the asset universe. They include clustering and network analysis.
 - c. Constraint generation: These let users easily define constraints. Some use prior and phylogeny statistics, others do not. Many require the user to define the sets of assets which let the library map the user's constraints to the data.
 - d. Uncertainty sets: These are used for robust optimisation, they ameliorate the sensitivity to estimation errors. They only cover expected returns and covariance.
 - e. Optimisation: These can either consume or internally carry out all aforementioned steps and produce a result with all the computed quantities. They can be used standalone or in a cross-validation framework (except the finite optimisers), where it is possible to tune the hyperparameters of the optimiser, including the entire processing stack.
3. Postprocessing: These just include reporting and visualisation. Which is by far the least mature aspect of the library.

It is worth noting that the preprocessing and processing steps can be bundled into a single pipeline step, which can also be used standalone or in a cross-validation framework, which can be used to tune the hyperparameters of the entire preprocessing and processing stacks together.

2.3 Prior statistics

`PortfolioOptimisers.jl` comes with a large number of moment estimation and prior statistics methods. It leverages `StatsBase.jl`'s covariance estimator definitions and API to allow interoperability with the Julia ecosystem, for example by composing with `CovarianceEstimation.jl`. They include a large number of methods, which can be combined in many ways. It's possible to compute the expected returns using a specific covariance method and vice-versa.

- Full or downside co-moments.
- Weighted, unweighted.
- Shrinkage.
- Probability-theoretic.
- Denoising, detoning.
- J-sparse.
- Regression-based.
- Rank based.
- Coskewness and cokurtosis.

The prior statistics fall under three categories:

- Frequentist: Empirical and timeseries factor models for low and high moments.
- Bayesian: Black-Litterman, vanilla, factor, Bayesian and Augmented.
- Information-theoretic: Entropy pooling, opinion pooling.

2.4 Phylogeny statistics

These bundle clustering and network analysis pipelines to explore the relational structure of the asset universe, and derive insights from it which can be used . Clustering and network analysis, though different tackle similar issues from different, complementary angles. The same analysis techniques can be used to exploit the structure each uncovers. As such we choose to categorise them under the same umbrella. The distance matrices include first and second order distances as defined in [3], and are compatible with every single covariance estimator that follows the `StatsBase.jl` API. The clustering and network analysis methods include:

- Hierarchical agglomerative clustering.
- Direct bubble hierarchy trees.
- K-mean clustering.
- Minimum spanning trees.
- Planar maximally filtered graphs.

2.5 Optimisers

`PortfolioOptimisers.jl` provides a large number of optimisers. They can be categorised into 4 types.

1. Naïve: Speed and robustness, but not necessarily optimal. Can be interesting as sub-optimisers to more complex optimisers, and as fallbacks.
2. `JuMP`-based: These are the most flexible and powerful, but they also require a solver, and are typically slower than other first-order optimisers. They support a wide range of constraints and objectives, and can be used to solve complex optimisation problems. They all use conic reformulations of problems.
3. Hierarchical: These utilise the relational structure of the universe in order to compute the risk of each group and sub-group, eventually leading to a diversified portfolio which encodes the relational structure as well as the risk characteristics of each group/subgroup. They are typically faster than `JuMP`-based optimisers, but support fewer constraints. They are very good for large numbers of assets and build well-diversified portfolios.

4. Meta-optimisers: These consume other optimisers and combine their results in order to produce a better result. They are typically slower than other optimisers, but can be very effective in finding good solutions.
5. Finite optimisers: These do not optimise portfolios in the traditional sense, but rather let the user specify a finite cash amount, as well as the asset prices and other options, and compute the best portfolio that can be constructed with the given cash amount. They are used after the others have produced a result.

2.6 Constraints and penalties

There is a huge variety of constraints. Every optimiser supports weight bounds constraints, all but the naïve ones also support fees. However, the richness of the constraints is in the [JuMP.jl](#)-based estimators. They can be divided into a few categories:

1. Upper/lower bounds: These limit the maximum and minimum weights of the assets in the portfolio. Their implementation differs depending on the optimiser, but in [JuMP.jl](#)-based estimators they are implemented as linear constraints.
2. Budget: These limit the total value of the weights of the portfolio. They integrate seamlessly with the finite optimisers in that they adjust the available cash based on the portfolio's budget. Together with bounds and linear constraints, they can be used to implement leveraged, dollar-neutral, and other types of portfolios.
3. Linear constraints: These are used in a variety of contexts, as risk contribution constraints, weight bounds, and relational structure constraints.
4. Cardinality constraints: These these can enforce sparsity, buy-in thresholds, inclusion/exclusion constraints. They can operate at the level of assets, or sets of assets. They are implemented as linear mixed integer constraints.
5. Fees: These penalise the return of the portfolio (thus indirectly penalise certain risk measures and objective functions). They include long and short proportional, long and short fixed, and turnover fees.
6. Weight penalties: These apply L-norm penalties to the objective function, or limit the L-norm of the weights of the portfolio. They can be used to regulate the sparsity and robustness of the portfolio.
7. Tracking and turnover: These limit how much a portfolio can deviate from a reference.

2.7 Design philosophy

`PortfolioOptimisers.jl` is implemented in Julia [11], using the `JuMP.jl` [12] mathematical optimisation embedded language, and leverages other well-established Julia libraries for much of its functionality. Aside from a strong commitment to FOSS principles, the library is designed atop four pillars:

1. Well-defined type hierachies. This lets us take advantage of generic fallbacks, and enables fearless extensibility, even at the user-level thanks to multiple dispatch.
2. Strongly typed, immutable structs. This lets the compiler aggressively optimise code, provides a single source of truth, and enables construction-time data validation that will not expire.
3. Compositional design. This lets us build complex objects from simpler ones, enables code reuse, and aids in extensibility.
4. Defensive programming. This lets us catch errors as early as possible, and provides a clear and consistent interface to the user.

With the exception of covariance estimators, which are defined in `StatsBase.jl`, every type in the library is a subtype of one of three types:

1. **AbstractEstimator**: These contain all the parameters to estimate a particular quantity. They are consumed by functions that perform the estimation. Estimators are used at user-level, they can be used by the public API to return a result. Estimators can be nested within each other, and can be used to build more complex estimators.
2. **AbstractResult**: Many of the estimators return more than one quantity, these are returned in a result type.
3. **AbstractAlgorithm**: These modify the behaviour of estimators via multiple dispatch. They are not used standalone or directly by the public API, but as part of an estimator.

The strict adherence to a well-defined type hierarchy lets developers and users alike extend and modify the library's behaviour without modifying the source code.

The code is available under an MIT license via the Julia General registry. It follows common software development best practices:

1. An extensive and high coverage test suite.
2. Automated documentation builds with an introductory user guide and deep examples, as well as a completely documented public and private API.
3. Miscellaneous code quality checks.

All as part of a GitHub continuous integration pipeline. It is under active development, and receives regular updates and improvements.

3 Equations

The template uses `i-figured` for labeling equations. Equations will be numbered only if they are labelled. Here is an equation with a label:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2} \tag{3.2}$$

We can reference it by `@eq:label` like this: (3.2), i.e., we need to prepend the label with `eq:`. The number of an equation is determined by the section it is in, i.e. the first digit is the section number and the second digit is the equation number within that section.

Here is an equation without a label:

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} \tag{3}$$

As we can see, it is not numbered.

4 Theorems

The template uses `great-theorems` for theorems. Here is an example of a theorem:

Theorem 4.1. (Example Theorem): *This is an example theorem.*

Proof. This is the proof of the example theorem. □

We also provide `definition`, `lemma`, `remark`, `example`, and `questions` among others. Here is an example of a definition:

Definition 4.2. (Example Definition): *This is an example definition.*

Question 4.3. (Custom mathblock?): How do you define a custom mathblock?

Answer 4.4. *You can define a custom mathblock like this:*

```
#let answer = my-mathblock(  
  blocktitle: "Answer",  
  bodyfmt: text.with(style: "italic"),  
)
```

Similar as for the equations, the numbering of the theorems is determined by the section they are in. We can reference theorems by `@label` like this: [Theorem 4.1](#).

To get a bibliography, we also add a citation.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Bibliography

- [1] H. Markowitz, “Modern portfolio theory,” *Journal of Finance*, vol. 7, no. 11, pp. 77–91, 1952.
- [2] D. Cajas, *ADVANCED PORTFOLIO OPTIMIZATION*. Springer, 2025.
- [3] M. M. L. De Prado, *Machine learning for asset managers*. Cambridge University Press, 2020.
- [4] P. P. DANIEL, *Portfolio Optimization: Theory and Application*. CAMBRIDGE University Press, 2025.
- [5] D. P. Palomar, “Portfolio Optimization.” [Online]. Available: <https://github.com/dppalomar>
- [6] D. Cajas, “Riskfolio-Lib (7.3).” [Online]. Available: <https://github.com/dcajasn/Riskfolio-Lib>
- [7] H. Delatte, C. Nicolini, and M. Manzi, “skfolio.” [Online]. Available: <https://doi.org/10.5281/zenodo.16148630>
- [8] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [10] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, “A rewriting system for convex optimization problems,” *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

- [11] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: [10.1137/141000671](https://doi.org/10.1137/141000671).
- [12] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, “JuMP 1.0: Recent improvements to a modeling language for mathematical optimization,” *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023, doi: [10.1007/s12532-023-00239-3](https://doi.org/10.1007/s12532-023-00239-3).

Appendix A

If you have appendices, you can add them after `#show: appendices`. The appendices are started with an empty heading = and will be numbered alphabetically. Any appendix can also have different subsections.

A.1 Appendix section

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.